

Rockchip CAN FD Developer Guide

ID: RK-KF-YF-162

Release Version: V1.3.0

Release Date: 2025-04-08

Security Level: ☐Top-Secret ☐Secret ☐Internal ☒Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED "AS IS". ROCKCHIP ELECTRONICS CO., LTD. ("ROCKCHIP") DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS, MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip's registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2025. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, software Boulevard Fuzhou, Fujian, PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

Product Version

Chipset	Kernel Version
RK3576	6.1
RK3506	6.1
RV1126B	6.1

Intended Audience

This document (this guide) is mainly intended for:

Technical support engineers

Software development engineers

Revision History

Version Number	Author	Modification Date	Change Description
V1.0.0	Elaine	2021-01-26	Initial version release
V1.1.0	Elaine	2024-03-22	Added RK3576
V1.2.0	Elaine	2024-07-22	Added RK3506
V1.3.0	Elaine	2025-04-08	Added RV1126B

Table of Contents

Rockchip CAN FD Developer Guide

1. CAN FD Driver
 - 1.1 Driver Files
 - 1.2 DTS Node Configuration
 - 1.3 Kernel Configuration
 - 1.4 CAN FD Communication Test Tools
 - 1.5 Common Command Interface for CAN FD
 - 1.5.1 Query Current Network Devices
 - 1.5.2 CAN FD Initialization
 - 1.5.3 CAN FD Transmission
 - 1.5.4 CAN FD Reception
 - 1.5.5 Common CAN FD Commands
2. Common Troubleshooting Issues with CAN FD
 - 2.1 Unable to Send or Receive
 - 2.2 Probabilistic Inability to Transmit and Receive
3. CAN FD Bit Rate and Sampling Point Calculation
4. CAN FD Speed Variation
 - 4.1 Loop Delay Testing

1. CAN FD Driver

1.1 Driver Files

Location of the driver files:

```
drivers/net/can/rockchip/rk3576_canfd.c
```

1.2 DTS Node Configuration

Main Parameters:

- `interrupts = <GIC_SPI 121 IRQ_TYPE_LEVEL_HIGH>;`
Completion of conversion generates an interrupt signal.
- `clock`

```
clocks = <&cru CLK_CAN0>, <&cru HCLK_CAN0>;  
clock-names = "baudclk", "apb_pclk";  
resets = <&cru SRST_CAN0>, <&cru SRST_H_CAN0>;  
reset-names = "can", "can-apb";
```

Clock properties for driving clock switches; reset properties for resetting the bus each time.

- `pinctrl`

```
&can0 {  
    assigned-clocks = <&cru CLK_CAN0>;  
    assigned-clock-rates = <200000000>;  
    pinctrl-names = "default";  
    pinctrl-0 = <&can0m0_pins>;  
    status = "okay";  
};
```

Configures can_h and can_l iomux for CAN functionality.

- `rx-max-data`

```
&can0 {  
    rockchip,rx-max-data = <4>;  
};
```

Configures the maximum number of bytes that can be received by rx. If only CAN frames are used, it can be set to 4 (rx byte supports 8), allowing the rx FIFO depth to be 64. If CANFD frames are used, it can be set to 18 (rx byte supports 64), resulting in an rx FIFO depth of 14 frames.

- `auto-retx-cnt`

```
&can0 {
    rockchip,auto-retx-cnt = <100>;
};
```

Configures the number of times tx will automatically retry after a transmission failure. If not configured, it defaults to no limit, continuously retrying. Configure a value between 1 and 0xffff, and the retries will stop after reaching the threshold, configure as needed.

- `dma`

```
&can0 {
    dmas = <&dmac0 20>;
    dma-names = "tx";
};
```

rx has DMA functionality, but if DMA is not desired, the above property can be removed.

1.3 Kernel Configuration

```
+CONFIG_CAN=y
+CONFIG_CANFD_RK3576=y
```

1.4 CAN FD Communication Test Tools

canutils is a commonly used CAN communication test tool suite, which includes 5 independent programs: canconfig, candump, canecho, cansend, cansequence. Here is a brief description of the functions of these programs:

`canconfig`

Use to configure the parameters of the CAN bus interface, mainly the baud rate and mode.

`candump`

Receive data from the CAN bus interface and prints it in hexadecimal form to the standard output, or outputs it to a specified file.

`canecho`

Retransmit all data received from the CAN bus interface back to the CAN bus interface.

`cansend`

Send specified data to a designated CAN bus interface.

`cansquence`

Automatically repeats incrementing numbers to the specified CAN bus interface, or specifies the reception mode and checks the received incrementing numbers.

```
ip
```

Configuration for CAN baud rate and functionality.

Note: The busybox also integrates the ip tool, but the one in busybox is a cut-down version. It does not support CAN operations. Therefore, please ensure the version of the ip command (iproute2) before use.

There are detailed compilation instructions for the above tool suite available online. If you are compiling buildroot yourself, simply enable the macros to support the above tool suite:

```
BR2_PACKAGE_CAN_UTILS=y  
BR2_PACKAGE_IPROUTE2=y
```

1.5 Common Command Interface for CAN FD

1.5.1 Query Current Network Devices

```
ifconfig -a
```

1.5.2 CAN FD Initialization

Disable CAN:

```
ip link set can0 down
```

Configure arbitration segment to 1M baud rate and data segment to 3M baud rate:

```
ip link set can0 type can bitrate 1000000 dbitrate 3000000 fd on
```

Print can0 information:

```
ip -details link show can0
```

Enable CAN:

```
ip link set can0 up
```

1.5.3 CAN FD Transmission

Transmitting (Standard Frame, Data Frame, ID:123, Data:DEADBEEF):

```
cansend can0 123##1DEADBEEF
```

Transmitting (Extended Frame, Data Frame, ID:00000123, Data:DEADBEEF):

```
cansend can0 00000123##1DEADBEEF
```

1.5.4 CAN FD Reception

Enable printing and wait for reception:

```
candump can0 &
```

1.5.5 Common CAN FD Commands

```
ip link set can0 down
ip link set can0 type can bitrate 500000 sample-point 0.8 dbitrate 2000000 dsample-point 0.8
fd on
ip -details -statistics link show can0
ip link set can0 up
echo 4096 > /sys/class/net/can0/tx_queue_len
candump can0 &
```

CAN Transmission:

dlc frame format test

Extended frame

```
cangen can0 -g 1 -e -I i -L i -D r
```

Standard frame

```
cangen can0 -g 1 -I i -L i -D r
```

Remote frame Standard frame

```
cangen can0 -g 1 -R -I i -L i -D r
```

Remote frame Extended frame

```
cangen can0 -g 1 -e -R -I i -L i -D r
```

CANFD No Bitrate Switching

Extended frame

```
cangen can0 -g 1 -e -f -I i -L i -D i
```

```
cangen can0 -g 1 -e -f -I r -L i -D r
```

Standard frame

```
cangen can0 -g 1 -f -I i -L i -D i
```

```
cangen can0 -g 1 -f -I r -L i -D r
```

CANFD BRS

Extended frame

```
cangen can0 -g 1 -e -f -b -I i -L i -D i
```

```
cangen can0 -g 1 -e -f -b -I r -L i -D r
```

Standard frame

```
cangen can0 -g 1 -f -b -I i -L i -D i
```

```
cangen can0 -g 1 -f -b -I r -L i -D r
```


2. Common Troubleshooting Issues with CAN FD

2.1 Unable to Send or Receive

Loopback Mode Testing:

```
ip link set can0 down
ip link set can0 type can bitrate 500000 sample-point 0.8 dbitrate 2000000 dsample-point 0.8
fd on loopback on
ip -details -statistics link show can0
ip link set can0 up
echo 4096 > /sys/class/net/can0/tx_queue_len
candump can0 &
```

In loopback mode, if `cansend` is followed by `candump` being able to receive, it indicates that the controller is functioning properly. Under this condition, it is only necessary to check: whether IOMUX is correct; whether the hardware connection is correct; whether the terminal has a 120-ohm resistor connected; whether the conversion chip is normal.

2.2 Probabilistic Inability to Transmit and Receive

Ensure that the bit rate is accurate; the following command can display the actual bit rate and configuration information of the CAN at present. If there is a deviation in the bit rate, it can cause anomalies in transmission and reception. It is necessary to adjust the input clock according to the bit rate to achieve an accurate bit rate.

```
ip -details -statistics link show can0
```

Adjust the sampling point; the aforementioned CAN command will print the currently configured sampling point. It is best to ensure consistency of sampling points across the same network, which can guarantee the stability of transmission and reception.

3. CAN FD Bit Rate and Sampling Point Calculation

The current CAN architecture automatically calculates based on the input frequency and bit rate. The sampling point rules follow the CiA standard protocol:

```

/* Use CiA recommended sample points */
if (bt->sample_point) {
    sample_point_nominal = bt->sample_point;
} else {
    if (bt->bitrate > 800000)
        sample_point_nominal = 750;
    else if (bt->bitrate > 500000)
        sample_point_nominal = 800;
    else
        sample_point_nominal = 875;
}

```

Bit rate calculation formula (detailed principles can be found on Baidu, only chip configuration related are introduced here):

$$\text{BitRate} = \text{clk_can} / (2 * (\text{brq} + 1) / ((\text{tseg2} + 1) + (\text{tseg1} + 1) + 1))$$

$$\text{Sample} = (1 + (\text{tseg1} + 1)) / (1 + (\text{tseg1} + 1) + (\text{tseg2} + 1))$$

brq, tseg1, tseg2 refer to the BITTIMING register in the CAN's TRM.

4. CAN FD Speed Variation

For speeds exceeding 5M, instability may occur. In such cases, it is necessary to first test the loop delay and then configure the TDC based on the loop delay.

4.1 Loop Delay Testing

Perform the following operations, and the value read back from the 0x2ac00110 register is the loop delay.

```

io -4 0x2ac00108 0x80
cansend 001#aaaaaaaa
io -4 0x2ac00110

```

After the loop delay test is completed, configure the TDC.

```

io -4 0x2ac00108 0x19

```

Configure the `tdc_offset` to half of the loop delay, enable `tdc_enable`, and refer to the TRM CANFD section for details on the RKCAN_FD_TDC register. The final TDC needs to be coded and written into the `tdc` configuration of the `rk3576_canfd_set_bittiming()` function.

It is also required that the sampling points on the CANFD variable speed network are consistent; otherwise, there may be anomalies in the reception of the BRS bit. For reference on sampling point settings:

```
ip link set can0 type can bitrate 500000 sample-point 0.8 dbitrate 2000000 dsample-point 0.8  
fd on
```